



The Open University

Mathematics and Computing: Level 3
TM354 Software engineering

TM354

Glossary

A

abstraction The process of describing the essential features of a problem and ignoring other details. It permits you to concentrate on a problem at some level of generalisation, without regard to irrelevant detail. It also permits you to work with concepts, terms and models that are familiar in the problem environment (or domain).

acceptance testing The formal testing of an entire system, conducted by a customer, to determine whether it satisfies its acceptance criteria and thus whether the customer should accept the system.

action An action is an atomic execution (contrast with activity) that results in a change in the state of the model. In an object-oriented context it can be realised by sending a message to an object or by modifying a link or the value of an attribute.

activation On a sequence diagram, a long rectangle, coincident with the lifeline of an object, showing the entire period when an object is active.

activity In UML, an activity is an ongoing non-atomic execution (contrast with action) within a state machine or within an activity diagram; in a given context, activities result in one or more actions.

In a software development process, it is a unit of work comprising one or more tasks that someone may be asked to perform.

activity diagram A representation of a workflow through activity states or action states and in which most or all of the transitions are triggered by completion of an activity in the source states; an activity diagram can be used to represent the workflows within the business domain.

actor A representation of users of a software system in a particular role, when interacting with use cases. An actor can also represent the role of an external system that interacts with a software system.

adapter A design pattern that addresses the problem of a class having an interface different from the one the client expects.

adapter pattern A design pattern used when a client expects to make calls to a particular interface different from the one that is implemented.

aggregation A relationship between an aggregate class and other classes. An aggregate object is an object which can be thought of as being composed of other objects (components), respectively, instances of the other classes. Some authors consider that the objects that are components in one aggregate object cannot be components in another aggregate.

agile An umbrella term used to describe a variety of methods that promote a set of practices that encourage simpler, more light-weight, faster and nimbler software development that can adapt to the inevitable changes in customer requirements. The continual realignment of development goals with the needs and expectations of the customer aims at software that better serves its purpose.

agile UP A version of UP where the initial disciplines (business/domain model, requirements, analysis and design) are collapsed into a single 'model' discipline and where iterations are short (weeks rather than months) and frequent; product is delivered incrementally if at all possible; design and implementation do not wait until all the requirements are found.

alt See fragment.

alternative fragment See alt.

ambiguity Open to several interpretations.

analysis The term analysis is often used as a generic term for the activities that precede design, the process of eliciting the requirements of a system from the user (requirements elicitation), and then generating the requirements specification document

that identifies (among other things) what the software should do (its functional requirements) and the environment in which it must work. In this module we use the term analysis to refer specifically to a set of activities that follow an initial requirements phase: modelling how the structure and behaviour of the system will meet its specification from a user's perspective. (See also requirements, design, implementation and testing).

analysis class A class that represents structural elements of an initial software solution. Unlike a concept (a domain class), it may define attributes but operations will be added later during design.

analysis model An initial representation of the structure of a software solution by a class diagram; it represents analysis classes, their associations and constraints.

analysis pattern A high-level pattern describing a solution at a conceptual analysis level problem.

analysis structural model See analysis model.

analytical completeness A measure of completeness where only the functional requirements are considered.

applet A small Java program that is downloaded from a server and executes in a container consisting of a Java plug-in installed in the client's browser.

Application Controller pattern A design pattern which addresses the concerns in the interaction between actors and a system, when the interaction takes place through user interfaces.

Architectural pattern Codifies recurrent software architectures by describing the key elements of the architecture and how they fit together. It also describes the qualities of systems that are assembled following the pattern.

architectural style See Architectural pattern.

architectural view A model of the architecture, that captures the concerns of a group of stakeholders of the system.

architecture See software architecture.

architecture specialisation The process of specialising an architecture by selecting specific components etc.

architecture specification The specification of the architecture of a system. It is also the third step in the product line process, where reference architectures are produced.

assertion In DbC, a precondition, a postcondition or an invariant.

assertion (handling) mechanism A programming construct which, at run-time, allows an assertion to be executed, and alerts the user to an assertion violation. Also referred to as an assertion mechanism.

assertion statement A statement of what must be true at a particular point in the execution of a method.

association A structural element that describes a set of links, in which a link is a connection between objects. It is a semantic relationship between two or more classifiers that involves the connections among their instances.

association class An association that also has class properties. Each instance of the association class defines values for those properties.

association loop invariant An invariant showing the relationship between two paths around a loop.

assume A condition that a component needs, to operate correctly.

asynchronous (communication) When the sender can continue without waiting for an answer from another component.

attribute A property of an object which is part of the object's state, and which can be changed. In object-oriented analysis and design, attributes generate accessor methods: one to 'get' the value of the attribute and one to 'set' or 'reset' its value.

B

baseline A configuration version singled out as special because it forms a foundation from which further development can progress, typically associated with the major milestones of a project.

basis-path testing A white box testing technique based on the cyclomatic-complexity metric which ensures that all statements in a method are tested at least once.

behaviour model See dynamic model.

bidirectional association An association that is navigable in both directions.

blackboard A variant of a data-centred architectural style in which the store is active and informs users of changes, so that communication may be initiated from either end.

black box testing Used to test that each aspect of the customer's requirements is handled correctly by an implementation. Black box testing 'sees' a system through its specification – the specification defines what the system should do and, through black box testing, we make sure it does it.

boundary-condition testing Unit testing in which test data are chosen to check that methods perform correctly at the extremities of input ranges.

boundary testing A type of black box testing that focuses on producing test data at the boundaries between partitions of the input data space (also known as fence testing).

Note that the input data space is simply the set of all values that an input can take, and a partition, also known as a subdomain, is simply a set of input values that require essentially the same processing.

bursty event An event coming in sudden and unpredictable clusters overlaying normally low background activity.

business event A business event is something that a system must respond to.

business modelling See domain modelling.

business process Definition of what gets done in the business: by whom, in what order, needing what and with what consequences.

business rule A constraint on how the business is run; that is, a constraint on the business processes.

business transaction A significant activity, episode, interaction, in a business.

C

callee Component that receives a procedure call in a call-return architecture.

caller Component that makes a procedure call in a call-return architecture.

call-return style An architectural style in which one component (the caller) makes a procedure call to another component (often known as the callee) and waits for the call to return.

Cascade pattern When designing an interaction diagram to implement a use case, there is a need for an object to send messages to another object with which it does not directly have an association. The Cascade pattern delegates responsibility to another object along a single thread of control (in contrast to the Fork pattern). (See also Law of Demeter.)

case analysis When carrying boundary testing, case analysis is the process used to partition test data into subdomains, i.e. sets of input values that require similar processing.

Case analysis aids the identification of boundaries between subdomains. At these boundaries, the kind of processing required changes.

CASE tools See computer-aided software engineering.

check(ing) in A part of the configuration management process where a configuration item is returned to the configuration repository after editing, usually with a new version number, and it is made available for others to check out.

check(ing) out A part of the configuration management process where a specific version (usually the latest) of a configuration item is removed from the configuration repository so that a developer can edit it.

circular dependency A series of two or more dependencies between different things (typically modules within a software system) that form a closed loop. If module A depends on module B which depends on module C, then a circular dependency arises if module C depends on module A.

class diagram A diagram that shows a set of classes, interfaces and collaborations, as well as their relationships. It is a representation of a structural (or static) model of a domain or of a system.

class invariant A constraint that must be true at all times, other than during when one of its methods is executing. A statement about the values of the attributes of a class that must hold for all instances of that class.

class method A method defining how a class responds to a message, rather than how instances of the class respond to messages. (See also instance method.)

class model See structural model. A class model is represented in a class diagram.

class-responsibility-collaboration cards An approach to identifying classes and their responsibilities using simply structured cards.

class-responsibility-collaborators model See class-responsibility-collaboration cards.

class variable In Java, a data field declared as static. (See also instance variable.)

client In software development, the term refers to the person (or company) who commissions and pays for a piece of software to be developed or to be purchased as a package (in this context the term client is synonymous with the term customer).

In programming, the term often refers to the method/class/object that uses the methods of another class/object (the supplier or server).

In network programming, the term refers to a program/process running on one computer that can obtain services from a server program running (usually) on another computer.

client-server An architectural style in which one component (the server) waits for requests from clients, processes each one as it is received, and returns a response to the client.

client-server style See client-server.

cloud See cloud computing.

cloud computing A model in which software runs on remote facilities provided by a third-party supplier.

cohesion A way of describing how closely the activities within a single module are related to each other. A cohesive module should, ideally, do just one thing. In software development, high cohesion is preferred to low cohesion as it results in a more flexible software system because each module performs clearly defined operations/tasks.

cohesiveness See cohesion.

commitment A move of the planning game that decides what to do next.

communication diagram A UML interaction diagram that focuses on the structural aspect (instances and links) of an interaction. In previous versions of UML a communication diagram was known as a collaboration diagram.

compiler A software tool that translates (compiles) a program written in one language (the source code) into a program in another language and, in so doing, identifies errors in the source code.

completeness The degree to which full implementation of the required functionality has been achieved.

complexity A complex system consists of a number of parts with a large number of relationships between them. A complex program is one that is difficult to understand.

component In general, this refers to an identifiable part of a system. For its use in connection with software systems, see software component.

component architecture An architecture specified in terms of components and the connections between them. It can be specialised by selecting appropriate versions of each component.

component-based development (CBD) An approach to the development of software by assembling reusable self-contained software components.

component collection The fourth step in the product line process where the repository of candidate components accompanying the architecture is gathered together.

component selection and specialisation The process of selecting appropriate components for a system and specialising them to the application domain. (See also domain.)

composite state A state that contains some internal behaviour, which can be represented with a state diagram. (Also referred to as a compound state.)

composition A form of aggregation with strong ownership; component objects of an aggregate have no life after the destruction of the aggregate.

computer-aided software engineering (CASE) The use of specialised application programs (tools) that provide some form of automated support during the development of a software system for the benefit of its eventual implementation. Consequently, it includes one or more methods, as well as their accompanying techniques and notations, used for activities such as analysis and design.

concept(ual) model The structural model that represents the static part of the domain: the concepts and their relationships.

concern Some aspect of the system that is of crucial importance from the point of view of one or more groups of stakeholders.

configuration All the modules (components) which together make up an executable software system. It is also a term for the collection of configuration items.

configuration item The configuration items are the items produced during the development and maintenance of software, which are stored, retrieved, changed, stored again, and so on. These include: use case diagrams and other UML diagrams; individual Java programs or classes, or the compiled versions of these; architectures and patterns; test plans and test cases.

configuration management The task of managing configuration items, which involves identifying the components of a continuously evolving system (the configuration) at discrete points in time, for the purposes of controlling changes to the configuration, and maintaining integrity and traceability throughout the system's lifetime.

configuration repository A library of configuration items managed by the configuration management system.

configuration version A consistent collection of versions of configuration items.

conformance testing Testing that the software product conforms to its requirements.

connector An element of an architecture that establishes the communication between components.

consistency The property of two descriptions agreeing with each other, that is, not making statements that cannot be simultaneously true.

constraint A limitation on or restriction of behaviour, or structure. In UML, a constraint expresses a rule that governs a model element or a combination of model elements.

construction In UP, phase when the final product is implemented.

constructor Used to initialise a new object. Technically it is not a method, though it is similar in structure. A constructor handles the dual tasks of allocating memory for the new object and ensuring that this memory is properly initialised. The programmer is expected to provide the code that will accomplish this initialisation. If not, a default constructor will be called that has no parameters and performs default initialisations.

In Java, a constructor has the same name as the class in which it appears. Constructors are not members; they are never inherited.

consumer A client in SOA.

container An object (or component) that exists to encompass other objects and provide operations to access or iterate over its contents. In Java EE, a special run time environment that provides components with supporting services (for example, life-cycle management, security, deployment, and run-time services).

context dependency A service required by a module in order for it to work.

contract A legal document describing the obligations of two parties entering into an agreement; important contract issues include confidentiality and intellectual property rights. In a software project, a contract exists between the customer (client) and developer (supplier) for the provision of a software product. Also, a way of describing what each task, activity or operation commits to achieve by stating the pre- and postconditions. (See also Design by Contract.)

controller In the MVC, the controller is the component that handles all user inputs which affect the model.

control-structure testing Unit testing in which test data is chosen to check that algorithms are coded correctly and to execute each statement in the code at least once.

copyright law The automatic legal protection afforded to an individual or organisation that publishes the expression of an idea, including software.

correct A correct system is one that meets all its requirements.

coupling A measure of interconnection among modules in a software system. In software development, low coupling is preferred to high coupling as it results in a more flexible software system. (See also encapsulation.)

coupling between objects (CBO) metric For a given class, CBO is defined as the number of relationships the class has with other classes. The more relationships a class has, and so the higher the value of this metric, the more difficult it is to understand the use of the given class.

CRC See class-responsibility-collaboration cards.

critical path A series of activities in a PERT chart which have no slack and therefore must be completed on time to ensure that the project is not late.

cultural requirement A non-functional requirement relating to the people involved in the product's development and operation.

customer See client.

cyclomatic-complexity metric A measure of the complexity of a method by counting the number of independent paths through a method body.

D

database (in data-centred) See repository.

data-centred Architectural style where there is a data provider which is a centralised store of persistent data.

data-centred style An architectural style in which there is a centralised data provider, together with a number of clients who are data consumers. If the provider simply responds to client requests it is called a database. A variant in which the data store informs clients when a change occurs is known as a blackboard.

data-flow See pipes and filters.

data-flow style An architectural style in which each component (called a filter) accepts a stream of data, processes it, and passes the result on to the next filter. A filter has no knowledge of other filters upstream or downstream but simply operates on the data it is given.

data lock-in The fact that each cloud service presents users with what is in effect a unique interface, making it difficult for a user to move their data and software from one cloud provider to another, as well as presenting problems in integrating with other systems.

data loss The amount of data lost because the system fails to record it.

DbC See Design by Contract.

decision nodes In an activity diagram a node that is used to indicate alternative transitions that can be made when leaving an activity; it is represented by a diamond shape. A decision node has a single incoming transition; the guards on outgoing transitions are mutually exclusive.

decomposition Splitting a problem into smaller and smaller parts or chunks until each one can be comprehended or dealt with by an individual. In software we identify a number of different activities or tasks that an individual or a group of individuals might perform. (See also partition.)

defect A verified lack of conformance to requirements.

defensive programming A design philosophy that says preconditions should always be checked. Each operation is responsible for making sure that it is safe to proceed. The checking is placed very close to the code that depends on it.

deliverable An output, such as a design document or source code, of some software development activity whose production gives evidence of the completion of the activity.

denial of use of service (or loss of access) Attack consisting of the denial of service to authorised users.

dependency injection A Java EE container service that creates and manages an EJB object 'injecting' the necessary references into the client.

deployment Configuring code to give a runnable system.

deployment model A UML model describing how the components of a system are to be mapped onto different machines and how they will communicate.

deployment view The identification of all the nodes that comprise a distributed software system, the components that are stored on each node and the objects that are stored there.

depth-of-inheritance-tree (DIT) metric For a given class, this is defined as its number of ancestors. Because of inheritance, a method in the given class can be defined in any of its ancestors. DIT is, therefore, a measure of the number of classes essential to the understanding of the methods of the given class. The deeper a class is in an inheritance tree, and so the higher the value of this metric, the more difficult the class may be to understand.

derivation A version of a configuration item based on other configuration items, which might have to be changed if the original item changes.

derived association In UML, an association that can be computed from one or more other elements. Although it adds no semantic information, it may aid understanding or be useful for design purposes.

derived attribute In UML, an attribute that can be computed from one or more other elements. Although it adds no semantic information, it may aid understanding or be useful for design purposes.

design The process of determining how to build an artefact that meets a set of objectives that are normally identified during analysis. It is one of the four basic technical activities within a software development process (see also analysis, implementation and testing).

Design by Contract (DbC) The process of developing software based on the notion of contracts between objects, which are expressed as assertions.

design decision Decisions taken during design that may impact on the quality of the software eventually produced.

design model A representation of the structure of a software solution by a class diagram; it represents classes, their associations and constraints and their responsibilities in terms of operations.

design pattern A middle-level pattern describing a solution at a software design level.

design structural model See design model.

developmental testing One of the four major types of software testing which checks that developmental activities have been carried out correctly, and ideally is performed by development staff and checked by quality assurance (QA) staff.

development process A set of rules that define how a software development project should be carried out. It incorporates a life cycle and involves a number of methods and procedures.

diagram In UML, a diagram is a graphical presentation of a collection of model elements.

disclosure Attack of private information or the unauthorised release of information.

discoverable The property of a service in a service oriented architecture that allows the architecture to identify services on behalf of a consumer. (See also service and service-oriented architecture.)

domain An area or field of knowledge characterised by a set of concepts and terminology that is understood by practitioners in that area.

domain analysis A form of requirements elicitation in which existing systems in the domain are examined and domain experts are consulted.

domain expert Someone who is considered to have a considerable level of knowledge or expertise in a given domain.

domain model An initial representation of the structure of a domain by a class diagram; it represents domain classes (concepts), and their associations.

domain modelling Modelling what already exists.

domain-specific knowledge A necessary but not sufficient element of competence made up of knowledge specifically concerned with the particular task or type of application.

domain structural model See domain model.

dynamic model Describes the behaviour of a software system over time.

dynamic modelling Modelling of behaviour as opposed to structure.

Dynamic Systems Development Method (DSDM) An agile software development method.

dynamic tool A tool that is used at run-time.

E

EJB container A run-time environment where EJB components are deployed.

elaboration In UP, phase most analysis and design occur.

element In UML, an element is an atomic constituent of a model.

encapsulation Hiding from the clients of a class the details of how the class is implemented. In other words, it is the process of hiding (making private) all the methods and data fields of a class used in the implementation of the public methods (or interface) of the class.

engineering A systematic approach to efficiently developing products which are fit for purpose, based on a body of knowledge.

enterprise architectures Software systems designed to meet the business needs of large organisations.

enterprise beans Another name for EJB components. They are server-side components, that is to say, they are distributed entities that provide remote services to clients distributed throughout the network.

Enterprise JavaBeans (EJB) A component technology, based on the Java programming language. It defines a standard that allows organisations to build their own components or acquire components from third-party vendors.

enumeration type A user-defined type with a finite number of literals.

error guessing An informal but successful black box testing technique in which the wildest test data that can be thought of is presented to the program unit.

error tolerance See robustness.

ethical decision A decision which may result in a living thing, including its environment, being adversely affected – emotionally, practically or morally.

ethical decision making Choosing between behaviour that is morally 'right' and that which is morally 'wrong'.

event In a Java system, an event is an action, such as the user clicking on a button; the system will respond to such an event by carrying out certain actions, provided the program defines an appropriate listener object for that event.

In operating systems, an event represents a signal to the CPU to interrupt its operation and switch context.

In analysis and design, events are elements of state transition diagrams which cause transitions between states.

In UML, an event is a specification of a noteworthy occurrence that has a location in time and space. There are four kinds of event in UML: call event: the event of receiving a call for an operation that is implemented by actions on transitions; change event: the event of a Boolean expression becoming satisfied because of a change to one or more of the values it references; signal event: an event that is the receipt by an object of a signal sent to it, which may trigger a transition between two states; time event: an event that denotes the satisfaction of a time expression, such as the occurrence of an absolute time or the passage of a given amount of time after an object enters a state.

executable assertion An assertion that has been coded and can be executed at run-time.

executable documentation Executable documentation is documentation that allows code to be automatically checked against documentation at run time. It should have no effect on program logic, and should be able to be switched on or off globally - so as not to impact performance. Not all aspects of documentation can be checked in this way, but Design by Contract and Test Driven Development are two different systematic approaches to executable documentation.

exploration A move of the planning game that finds out new things that can be done.

extension A form of inheritance in which the child class adds functionality to the parent class but does not change any inherited behaviour.

extension points A place in the main success scenario of a use case where the condition that triggers a subsidiary use case is tested.

external event An event that links a system to its actors, such as the pressing of a button or the sounding of an alarm. (Contrast with internal event.)

extreme programming (XP) An agile software development method.

F

factory A specialised object for creating and initialising objects needed by clients.

factory method A variant of the Factory pattern particularly useful in the definition of frameworks in which application-specific factories and products can be defined based on the interfaces defined in the framework.

Factory pattern A pattern to define a specialised object for the creation and initialisation of other objects.

filter Element in a pipe and filter architecture that takes some data and processes it to produce a result.

final state In a state diagram, a special mark (bull's eye), with only incoming transitions, indicating where processing terminates.

fit criteria Plural of fit criterion.

fit criterion A quantification or a measurement of a requirement expressed in a way that allows you to assess whether or not the final product satisfies, or fits, that requirement.

fixture The state in place and the data to be used on a test case.

flexibility The property of a software system that indicates how easily the system can be modified.

foreign key In a database table, a column that holds a key from a different table.

fork In activity diagrams, a fork node splits control into multiple parallel flows.

Fork (pattern) When designing an interaction diagram to implement a use case, there may be a need for an object to send messages to another object with which it does not directly have an association. The Fork pattern centralises control in the sender (in contrast to the Cascade pattern). (See also Law of Demeter.)

formal technical review (FTR) A process to critique a product of the software development process with a view to improving it. The product might be a customer requirements document, a piece of code, or a test or maintenance plan.

forward engineering The process, usually automated or semi-automated in a CASE tool, for producing programming language source code from a model. If the code is automatically generated it usually requires additional human input before it is complete and can be machine compiled or interpreted.

fragment For our purposes, a way of encapsulating behaviour inside a labelled box. The label can be, for example, opt, alt or loop, standing for optional, alternative and looping behaviour. Sometimes referred to as a frame.

frame See fragment.

framework A collection of classes that together provide behaviour common to a variety of applications. The programmer creates a new application by extending and, perhaps, overriding the behaviour of the classes contained in the framework. Also, the part of an architecture that can be reused.

freeware A piece of software that is given away free to users, although the copyright remains with the originator and the original software cannot be distributed by anyone else.

freezing A stage in change control where any further changes to a baseline are forbidden, which means disabling further check in operations for the items in the corresponding configuration.

functional (or logical) view In a software architecture it describes the system's main functional elements, their responsibilities, interfaces and primary interactions. A functional view is the cornerstone of most architectural descriptions.

functional requirement a specified action that a system has to perform

further analysis The adding of additional detail to a system specification. In particular, system operations and model constraints.

G

Gantt chart A project management chart named after its originator, the American management consultant Henry Laurence Gantt, in which the horizontal direction represents time and the vertical direction represents activities, and which can be set out as a table, whose rows show when the work takes place, or as a bar chart, whose horizontal bars show when the work takes place. Gantt charts emphasise the times at which things happen.

generalisation A relationship between classes in the object-oriented model that captures the fact that one class of objects is more generalised than another, that is, the methods and data fields of the generalised class (superclass) are a subset of the specialised class (subclass).

General Responsibility Assignment Software Patterns (GRASP) A collection of patterns which may help the designer assign responsibilities in commonly recurring design scenarios.

generic collection In Java, collection types can be parameterised by the type of the objects they contain.

glossary A list of terms or words relating to a particular domain or text, along with an explanation of each term.

grammatical parse Used in this module to refer to process for obtaining list of nouns from a written problem description to identify candidate classes.

GRASP See General Responsibility Assignment Software Patterns.

GRASP Creator A pattern that provides guidance on which class should be responsible for creating new objects.

GRASP Expert A pattern that provides guidance on which class should be responsible for initialising or changing the state of an object.

guarantee A condition that a component promises to other components.

guard A condition that must be satisfied in order to enable an associated transition to fire within a state machine or activity diagram.

guillemets Angled brackets (« ... ») used in UML to show a stereotype. They have other more general uses.

H

hacking In the popular media, the term is synonymous with computer misuse. In the computing community, a hacker is one who programs enthusiastically (even obsessively) or who enjoys programming rather than just theorising about programming.

hardware independence The degree to which the software is decoupled from the hardware on which it operates.

heroic programming A form of software development where a single person attempts to have in mind everything concerning a given problem and its eventual solution.

heuristic evaluation User interface code should always be user tested. However, heuristic testing allows preliminary testing of interaction designs without users. Heuristic evaluation involves analysing interaction designs against recognised checklists for good practice ('heuristics').

HTTP (Hypertext Transfer Protocol) A protocol for communication of data in the Web.

human factors The discipline that deals with the relationship between people and technology.

hybrid cloud A cloud offered by an organisation that combines a private cloud with externally provided cloud services.

I

idiom A pattern that applies within a programming language.

ilities Quality attributes, for instance security, reliability, availability, usability etc.

implementation The process of building a coherent set of instructions for a computer, that will cause it to exhibit a behaviour specified by the artefacts which are normally devised during design. It is one of the four basic technical activities within a software development process (see also analysis, design and testing).

implicit invocation See notification.

inception In UP, phase when the initial planning of development occurs.

inclusion A relationship between two or more use cases, when there is an overlapping area of functionality – that is, a common task – that can be factored out as a distinct use case. The new use case can then be used by each of the original use cases, so avoiding duplication.

increment The difference between two releases of a (software) deliverable at the end of successive iterations.

incremental development This development approach partitions the system into a number of independent or reasonably independent 'chunks'. The process begins by developing one chunk – the core – and adding successive increments to it over time. A significant point about this method is that the core is a functioning item of software that meets some of the requirements. As each increment is added, the resulting software is still functional.

independent components Architectural style where components execute concurrently and are decoupled as far as possible, but can communicate by messages that allow them to exchange data or coordinate their operations.

independent components style An architectural style in which components are largely decoupled and execute concurrently, but coordinate with one another or exchange data by means of asynchronous messages.

inertial convention The assumption that state components not explicitly described as having changed in a postcondition will remain unchanged by a method call.

inheritance The consequence of a generalisation/specialisation relationship; a subclass inherits the features of the superclass.

initialisation testing See start-up testing.

initial problem statement An initial statement of a problem from which development proceeds. In order to draw up detailed requirements it will usually be necessary to clarify and expand on the information in the initial problem statement during requirements elicitation. (See also requirements elicitation.)

initial state In a statechart diagram, an initial state is a way of indicating the default starting state. An initial state is not a true state; it is a marker with only one outgoing transition to the first state an object finds itself on creation. A statechart diagram must show no more than one initial state.

instance method A method that describes how a particular operation is carried out by every instance of a class. It contains the code that determines how every instance of a class responds to a particular message. (See also class method.)

instance variables A data field that is common to all the objects (instances) of a class but the value of which is specific to each object. This contrasts with a class variable (a data field defined as static in Java), whose fixed value is shared by all the instances of a class.

instrumentation The degree to which the system monitors its own operation and identifies errors that occur.

integration An activity in which individual software modules or components are combined into an executable whole.

Integration and release The process of bringing items together into a unified whole, and then releasing to a customer.

integration testing The second stage of developmental testing in which previously tested units are tested together as an integrated whole.

integrity In general, the property that data has not been subject to unauthorised change. In particular, a software quality factor affecting product operation requirements which determines the extent to which access to data by unauthorised persons can be controlled, that is, how well the data is secured.

interaction diagram A UML diagram that shows the sequencing of messages in an interaction. (See also sequence diagram and communication diagram.)

interface In general, this refers to the operations (methods), including their syntax and semantics, that a software system/subsystem/module provides to the clients of the system. In the object-oriented approach, the public operations associated with an object are referred to as its interface, or its interface operations. In Java, an interface is a construct similar to a class, which provides a specification (in the form of method headings but no implementation) of the methods for which classes implementing the interface must provide implementations.

interface testing Unit testing in which test data is chosen to check that the flow of information into and out from an object of the class is correct. Also a form of system testing, in the case where the system communicates with other systems in the outside world, that tests its interaction with the communication system; in this context it is also referred to as protocol testing.

internal event An event that links the objects within a system, such as the throwing of a language exception. (Contrast with external event.)

internal transition A special transition that does not involve a change of state. An internal transition does not cause the execution of any entry or exit actions.

interoperability A software quality factor affecting product transition requirements which determines the effort required to couple one system to another, that is, how easy it is to interface the system with another, should the customer require it.

invariant See class invariant.

iteration A distinct sequence of activities within a life cycle that may be repeated. Within a development process, the result is a (software) deliverable that conforms to certain criteria.

iteration planning game A planning game for an iteration, played even more frequently, on tasks instead of stories; the players are the programmers.

iterative and incremental development This development approach starts with a subset of the requirements and incrementally grows the system with further increments. Each increment is developed over several iterations; an iteration is a complete small project, with a short timeframe, consisting of analysis, design, implementation, testing and integration, and results in a partially working system.

iterative development This development approach continuously repeats a set of activities improving the outputs each time at the feedback received from a previous iteration.

J

Java Enterprise Edition (Java EE) A framework for constructing enterprise systems.

java platform, enterprise edition See Java Enterprise Edition.

jitter The amount of variation in latency, for example, are some responses very quick but others take much longer?

join In activity diagrams, a join node synchronises multiple parallel flows.

L

lack-of-cohesion-in-methods (LCOM) metric The number of pairs of methods that make reference to the same attributes minus the number of pairs of methods that do not; or zero should this be negative. In highly cohesive classes, methods will manipulate the same attributes.

latency The time taken to process a stimulus, for example, how long does a search engine take to respond to a query?

Law of Demeter A design principle governing the flow of messages among objects. It says that in response to a message m, an object O should send messages only to the following objects:

- O itself;
- objects that are sent as parameters to the message m;
- objects that O creates as part of its reaction to m;
- objects that are directly accessible from O (using values of the attributes of O).

law of patents The law that relates to the protection of inventions.

layer A logical partitioning of a system.

layered architecture An architecture organised using a Layers pattern.

layered style An architectural style in which the system is structured as a series of layers, each layer communicating only with the layers immediately above and below it.

Layers pattern This organises the large-scale functional structure of a system into discrete layers of distinct, related responsibilities, with a clear separation of concerns between the layers.

legacy system An old software system that is still required because it meets some useful purpose to an organisation. It is not considered to be replaceable and so it is maintained, usually on a restricted budget.

legal contract A legal contract sets out the parameters of an agreement in a legally binding way. Breaking such a contract can result in legal action being taken.

legal requirement A non-functional requirement about laws and standards that apply to the product.

life cycle The sequence in which a number of different activities are carried out to define an overall process for the development of a software system.

lines-of-code (LOC) metric A very simple measure of the complexity of a system description is given by counting the number of lines in a piece of code.

link An instance of an association that shows a semantic connection between objects.

location transparency In a service-oriented architecture (SOA) it allows consumers to make use of the service without being aware of where the service is actually located, i.e. on which machine or even in which business organisation. The requester of the service and the service itself are only loosely bound; in particular, the requester has no

knowledge of the service implementation or its platform, and they only communicate through messages. It is also possible for components to be location transparent and be invoked using a logical name rather than a physical address.

locator See registry.

look and feel requirement A non-functional requirement about the product's appearance.

loop See fragment.

loop fragment See loop.

loop testing A white box testing technique that is used to test loop constructs to ensure they execute the right number of times.

M

mailbox semantics A way of getting two active objects to communicate. The sender sends a message and then carries on processing, regardless of what the receiver is doing. The receiver, working at its own speed, can then read waiting messages and process them.

maintainability This is a software quality factor affecting maintainability and portability requirements, which determines the effort required to understand, locate and fix errors in a system, that is, how easily bugs can be found and fixed.

maintainability and portability requirement A non-functional requirement about expected changes to a system and allowed time to make them.

maintenance An activity in which the artefacts or products of a software development process are changed. It includes all changes made to a software system after it has been released.

major defect A defect which, if not corrected, would either cause a malfunction or prevent the attainment of a required result.

malleability Characteristic of software as being easy to change. Malleability creates a constant pressure for software to be changed rather than be replaced. Every change that is made to the software introduces the possibility of new errors.

Mean Time To Change (MTTC) The average of the time it takes to analyse a bug report, design an appropriate modification, implement the change, test it, and distribute the change to all users. In general, the lower the MTTC (for equivalent types of changes), the more maintainable the software product is.

merge node In an activity diagram, this is a node that brings together multiple alternative flows; represented by a diamond shape; a merge node has a single outgoing transition.

message-driven beans EJB beans that support asynchronous processing through message passing.

message number(ing) A means of identifying the ordering of messages in a flow of control. On an interaction diagram, the messages may proceed in a nested or hierarchical fashion. Ambiguity in the message flow is removed by using a multi-stage decimal for each message in an interaction. Thus each message always contains as its prefix the number of the message that gave rise to it. If an object receives a message prefixed 2.6, all messages it sends until it has finished processing 2.6.1, 2.6.2 and so on (this is known as decimal message numbering).

method In object-oriented terminology, it is an operation that manipulates an object in some way.

In Java, a method is a member of a class and has a method heading (which incorporates its signature) and a method body.

In software development, it is a collection of integrated techniques, with their associated notations and rules, that takes a set of predefined inputs and converts them to a set of predefined outputs.

milestone In project management, the points where key deliverables are completed, reviewed and then frozen.

minor defect A defect which, if not corrected, would neither cause a malfunction nor prevent the attainment of a required result.

miss rate Measures the proportion of events that are not responded to.

model A simplification of reality, created in order to understand an aspect or viewpoint of the system being created. In the MVC, the model the component that does the processing.

modelling A way of thinking about things and ideas in the real world. A model is a way of expressing a particular view of an identifiable system.

modelling language A way of expressing the various models produced during the development process. A modelling language defines a collection of model elements, which are roughly equivalent to the utterances in a spoken language. (A model is made up of model elements, as sentences are made up of words.) UML is a modelling language.

Model–View–Controller (MVC) pattern Describes a high-level solution to the general problem of human–computer interaction, relating what the user sees on a screen and does with a keyboard, mouse or other interaction device, to the functions of an application and the data that it stores. The ‘model’ is the internal workings of the program, the ‘view’ is how the user sees the state of the model and the ‘controller’ is how the user changes the state or provides input. Whenever some input through a controller changes the primary data of the model, the model then informs all views and all controllers so that they in turn can update their representation of the data.

modifiability See flexibility.

modification (or loss of integrity) Attack consisting of the unauthorised modification of data.

modularisation See modularity.

modularity The division of a software system into separately named and addressable components, called modules, that are integrated to meet the requirements. Modularity is a basic attribute of software that allows the partition of a set of requirements into a number of intellectually manageable subsets.

module A program unit (or component) that is used exclusively through an external interface. It is important for enabling a system to be comprehended at any required level of detail. A software system is a collection of modules.

morality The degree of conformity to a set of principles that determine ‘rightness’.

MoSCoW A scheme in DSDM classifying requirements into Must have, Should have, Could have, and Won’t have.

multiplicity A specification of how many objects of one class are related to one object of another class.

N

namespace A boundary within which names are unique.

navigability Refers to the ability to traverse one or more associations to obtain the object or set of objects associated with an instance of a class through those associations.

navigation expression A description of the path taken to traverse the links (corresponding to one or more associations) from one object to another object (or set of objects) or to its (or their) attributes.

non-deterministic A situation where one of a number of outcomes is possible, for example when throwing a dice.

non-functional requirement A quality that a system must have.

notification See publish-subscribe.

notification style An architectural style in which one kind of component, the observer, registers itself with another component, the subject, in order to be notified when some particular event occurs at the subject's end.

number-of-children (NOC) metric For a given class, NOC is defined as the number of children for that class. This metric is a measure of the number of classes that will be affected by changes to a given parent class.

O

Object Constraint Language (OCL) A text language for specifying constraints, Boolean expressions and queries. OCL can be used to construct expressions for constraints, guards, actions, preconditions, postconditions and assertions. OCL is part of UML.

object diagram An instance of a class model that shows objects (possibly value of attributes) and their links at a point in time.

object model A collection of objects and their links, where every object in the model is an instance of a class, and each link is an instance of an association in the corresponding class model. An object model is represented by an object diagram.

observable In the Observer pattern the object, also called the subject, that keeps objects registered with it informed when its state changes.

observer In the Observer pattern objects register themselves with a Observable, in order to receive notifications when the latter's state changes.

Observer pattern A design pattern that addresses the issue of supporting dependencies between objects, whilst keeping coupling low.

OCL See Object Constraint Language.

open-source software Similar to freeware, except that it is maintained and upgraded openly by a community of expert users; code can be modified.

operability The ease of operation of a program.

operation See method.

operational and environmental requirement A non-functional requirement about how the environment on which the product will have to work (under water, for example), and what considerations must be made for this environment.

operator testing See user-command testing.

opt See fragment.

optional fragment See opt.

oracle The criteria in a test case that helps define whether the test is passed.

organisational units The groups to which people belong.

P

package UML employs the concept of a package to enable the developer to modularise a given system or part of that system. A model comprises a set of packages that hold model elements, such as classes and use cases. A package may contain other packages.

partition The decomposition of a problem into a number of sub-problems that are independent of each other.

partitioning This black box testing technique focuses on producing test data at the boundaries between partitions of the input data space (or input domain). Partitioning is based on the observation that common errors are often caused by a data item being just one out, or a loop being executed one too many or one too few times, for example; such errors are most visible at the boundaries of the input data space. (Also referred to as equivalence partitioning.)

path parameter Part of a path to be replaced by the corresponding element in the actual URI the client uses when making a request.

peer-to-peer An architecture where there is no clear distinction between clients and servers.

peer-to-peer style An architectural style in which all components are both clients and servers and any component can request services from any other component.

performance The ability of a system to process service requests.

performance requirement A non-functional requirement about how fast, how safe, how accurate, how available and how reliable the product must be.

performance testing System testing that tests that the system meets all specified operating requirements for speed, number of concurrent users permitted, etc.

PERT chart A project management chart in which PERT stands for Project (or Program) Evaluation and Review Technique. It shows the activities as boxes with lines joining them to indicate the interdependencies and the flow of critical information. PERT charts emphasise interdependencies and allow critical path analysis.

pipe Element in a pipe and filter architecture that is a conduit along which the data flows.

pipes and filters An architecture where filters are the elements that take some data and process it to produce a result, and pipes are the conduits along which the data flows. Also known as data-flow.

placeholder Used in this module to refer to the name of a type that is being used as a stand-in until there is sufficient information to make an informed decision.

plan-driven development Term used to distinguish traditional, more prescriptive approaches to software development from agile development approaches.

planning The first stage of a formal technical review.

Planning Game An XP practice with two participants, the business and the development. This game takes place not only at the beginning of the project but at regular intervals (say every three weeks) to determine the priorities and work for the next release and to define a plan. It is played with story cards. (See iteration planning game.)

polymorphism In object oriented programming, when a program specifies that some message is to be sent to some object, it is not always possible at compile time to determine which of several candidate methods should correspond to the message. This is because, for inheritance or other reasons, there may be several classes of object with a common interface, and the identity of the class of the receiver may not be determined till run time.
Such a message is said to be polymorphic.

portability A software quality factor affecting product transition requirements which determines the effort required to transfer the system from one hardware platform and/or software environment to another, that is, how easily the system can be used on another machine should the customer change their platform or should other customers require it.

postcondition A constraint that must be true at the completion of an operation, or use case.

precondition A constraint that must be true when an operation is invoked, or when a use case is carried out.

predefined type A data type that is provided by a programming or a modelling language and available to the developer. Examples are data types for numbers, strings and collections.

preferred version In configuration management, the default version which is checked out, usually the latest.

presentation layer In a layered architecture, the layer concerned with the user interface.

primary key In a database table, a column that uniquely identifies each row of the table.

private cloud A cloud that belongs to a particular organisation, usually one large enough to generate economies of scale internally.

problem domain See domain.

procedure In general, a procedure is a prescribed set of steps for carrying out a task. In programming, it refers to a block of code that carries out a specified programming task and may or may not have input and output parameters. In object-oriented systems, methods are used to implement procedures.

process In program execution, a process is a dynamic (abstract) entity that executes a program on a processor. It is associated with resources such as a main memory and files. In this context, processes are also sometimes referred to as tasks.

In software development, a process is a set of activities or operations that takes one or more inputs, such as a design document, and results in one or more products, such as a software module and its test cases.

process improvement The idea that an organisation should learn from difficulties encountered with their existing processes, and improve as a result.

process model See life cycle.

process-related human factors Factors affecting the performance of a software engineer which arise either from the characteristics and skills of the engineer or are external influences that have an impact on how effectively they can apply themselves to the task. They include person-related human factors, task-related human factors, and environment-related human factors.

process view In a system architecture, this view addresses concurrent aspects of the system at run-time, including all system processes and system start-up and shut-down.

product In the Factory design pattern the object the factory produces.

product backlog An artefact in Scrum (an agile method) that consists of an ordered structured list of what needs to be done to a system.

product line A set of related software products for different customers, reusing major parts of the software from customer to customer, with only those parts peculiar to the particular customer being different.

product-line initiation This is the first step in the product line process, where it is recognised that there is a class of commonly recurring software systems where solutions do not differ much from each other.

product operation requirements How a product will be used

product-related human factors The human factors involved in the people–technology relationship.

product revision requirements How a product will be maintained.

product transition requirements How a product will be modified for different operating environments.

profession 'A form of employment, esp. one that is possible only for an educated person and after training (such as law, medicine or teaching) and that is respected in society as honourable.' (Longman Dictionary of Contemporary English, 1987.)

project management The activity concerned with ensuring that a software project is completed within the estimated time. It involves determining the work involved, assigning work units to people or teams for periods of time, and giving a work plan for the project.

project plan A plan breaking down the development of a system into stages and showing how much resource is required at each stage.

protocol testing See interface testing.

prototype A simplified software system that is a subset of the eventual product.

prototypical interaction An interaction described using some prototypical objects.

prototypical object An object representing the general case rather than a specific instance.

provided interface The set of operations a component makes available to other components.

provider In SOA the platform on which a service is implemented.

public cloud A cloud such as Amazon EC2 or Google App Engine which is available to anyone willing to purchase the service.

public-domain software Similar to freeware except that the originator has signed away the copyright. This means that users can modify the software, redistribute it or incorporate it into their own work.

publisher Component, in a publish-subscribe architecture, that notifies other components of updates.

publish-subscribe Architectural style where subscribers register to receive updates (often messages) whenever a publisher posts a new item.

Q

qualified association An association that uses attributes of one class to partition the set of objects related to an object across the original association; those attributes are the qualifier of the association.

quality attribute See non-functional requirement.

quality attribute scenario A tool which allows specification of non-functional requirements in a more precise and detailed form which can be used to help make architectural decisions.

quality management An activity that takes place throughout the software development process to ensure the quality of the final product.

quality management system (QMS) An organisation-wide mechanism for building quality into projects and for managing the quality control process. Consists of the managerial structure, responsibilities, capabilities and resources that ensure that software products developed by projects will have the qualities desired by both the customer and the developer.

quality manual A document containing quality standards, guidelines and procedures.

quality plan A document specifying the quality controls for a project.

Quantity pattern A pattern which provides useful guidance on how to represent dimensioned quantities, that is, values plus their units of measurement.

R

random testing A black box testing technique in which test data is randomly generated within the input data space.

realisation An implementation of an interface.

record A row of a database table.

recursive A recursive association is one between a concept/class and itself.

refactoring The reorganisation of code structure for reasons such as clarity or maintainability, while making no change to the external behaviour.

registry A mechanism that allows clients to find services.

regression testing One of the four major types of software testing, which occurs during developmental testing and system maintenance, and which checks that fixing one bug has not introduced others.

release A configuration version delivered on completion of the project for use by the customer.

reliability A software quality factor affecting product operation requirements which determines the likelihood with which a system can be expected to perform its intended function, that is, how well the software does what it is supposed to do.

rendezvous semantics A way of getting two active objects to communicate. The sender indicates a desire to talk and the receiver indicates a desire to listen. Whichever of these happens first causes that thread to block until the other has happened, at which point communication takes place.

repository Centralised store of persistent data in a data-centred architecture.

repudiation Attack consisting of (false) claim by legitimate user that a message was not sent or received.

required interface The set of operations a component needs from other components.

requirement A desired feature, property or behaviour of a software system.

requirements See requirements engineering.

requirements analysis An activity undertaken to decide what needs to be done to solve the problem that has been articulated during requirements elicitation. The result is a deliverable that forms the basis for a solution to be implemented. It is expressed in technical terms that are understandable to the developers.

requirements-based testing One of the four major types of software testing which checks that a system meets the customer's requirements, and is carried out by QA staff in conjunction with the customer.

requirements documentation An activity undertaken to record agreed requirements.

requirements elicitation An activity undertaken to discover, identify, review, articulate and understand the users' needs and constraints on the software and its development. The result is a clear statement of the problem to be addressed.

requirements engineering An activity in the process of developing software that aims to elicit and specify the requirements of the client(s)/user(s) in a form that can be used in the design of the software and in assessing its quality once it is built. It can be divided into two main activities: requirements elicitation and requirements analysis.

requirements negotiation An activity undertaken to agree requirements.

Requirements pattern A description of knowledge used in the elicitation of requirements.

requirements specification A detailed description of the requirements for a system.

requirements validation A careful check of a requirements document, usually following a checklist.

resource balancing/levelling The process of juggling the assignment of resources to activities over time so that each resource is used at its capacity, and each activity receives the resources that it needs.

response-for-a-class (RFC) metric For a given class, RFC is defined as the size of the response set for the class. Classes with high values for this metric will be more difficult to understand than classes with low values.

response set All the methods of a class, including those inherited, and those it invokes.

restart testing System testing that tests the ability of the system to recover from errors of internal state.

RESTful A web service that uses a uniform interface consisting only of a set of standard methods which are always the same for every service.

reusability A software quality factor affecting product transition requirements which determines the extent to which a system (or system component) can be reused in other

applications, that is, how easy it is to reuse some of the software to make future developments more cost-effective.

reverse engineering The process, usually automated or semi-automated in a CASE tool, for deriving a model from source code alone.

review A software inspection meeting to confirm and list major defects, and to determine who will be responsible for addressing each defect.

risk management Analysis of risks in a development process and determination of steps to reduce them.

robustness The degree of immunity from damage when a program encounters an error. Also referred to as error tolerance.

role The behaviour of an entity participating in a particular context. It may also refer to the behaviour of people in a domain.

role name The identification of a specific role for a class in a given association.

S

scenario A specific sequence of actions that illustrates how a task can be performed. It is also a textual description of a specific instance of a use case or interaction with a user interface.

Scrum An agile software development method.

security A term used in connection with controlling access to the data stored in computers and in connection with the availability of mechanisms that control or protect programs and data.

security requirement A non-functional requirement concerned with aspects of a product such as security and confidentiality.

security testing System testing that tests that the system does not offer opportunities to breach security.

self-consistency System descriptions are said to be self-consistent if they make no self-contradictory statements.

self-documentation The degree to which the source code of a program provides meaningful documentation.

self-transition In a state machine, this is a transition that originates in one state and returns to the same state. They are commonly used to model loops or cycles.

sequence diagram A UML interaction diagram that focuses on the time-ordering of the messages.

server See supplier.

service A unit of reuse corresponding to a piece of functionality, described in a standard language, with published interfaces through which the service execution can be requested.

An abstract description of some unit of business functionality, usually described in terms that are meaningful from both a business and a technical perspective. (See also service-oriented architecture.)

service consumer (or consumer) Component accessing a service.

service description Specifies how a consumer can interact with a service.

service level agreement (SLA) A statement of the assume-guarantee contract of the cloud service.

service orchestration Coordination of services invocation.

service-oriented architecture (SOA) SOA is an approach to build enterprise systems that deliver application functionality either as services to end-user applications or to build

other services. In this approach, services are software assets, which are described and interact in a standard way, so that the focus of application development is the assembly of services as building blocks.

service-oriented cloud computing architecture (SOCCA) An architecture to address the integration of the principles of SOA with those of cloud computing.

service-oriented style An architectural style in which set of service providers makes services available to a set of service consumers. Consumers can combine services in order to carry out business processes.

service provider (or provider) Component implementing a service.

service registry (or registry, or locator) Allows a consumer to find a service.

service requester (or requester) See service consumer.

session beans A session bean represents a single client inside the application server. To access an application that is deployed on the server, the client invokes the session bean's methods. The session bean performs work for its client, shielding the client from complexity by executing business tasks inside the server.

shareware Software that is intended to be trialled for a limited period (often 30 days). Users are expected to register and pay any appropriate fee after this time has elapsed.

signature In general, the combination of name, parameters and return type of an operation. In Java, the combination of name and formal parameters in a method heading (that is, excluding method's return type).

simple state A state that contains no substructure. (See also composite state.)

singleton A design pattern used where there should be only one instance of a class.

Singleton pattern A pattern to ensure that a class has only one instance, and to provide a global point of access to it.

size (of software) This can be measured in terms of lines of code, number of symbols, size of source or object files etc.

snapshot A representation (using a UML object diagram) of objects and their links at a certain instant in time.

socio-technical systems A system seen as a whole, including the software, the hardware where it is installed and running, and the wider context that involves other technical components, but also people, organisations and other social structures.

software architecture '... the set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both.' (Bass, Clements and Kazman, 2012.)

software component In the context of testing, the minimal software item for which a separate specification is available. In the context of reuse, usually a routine, a class (consisting of a collection of routines), or a collection of classes (a package), which has a specification (including interfaces) and a list of its underlying principles and concepts, as well as quality information such as levels of proof and testing.

software development The process whose end product is a software product.

software engineer A person employed to undertake software engineering.

software metrics Measurable quantities that can be used to infer values for the software quality factors.

software quality At a general level, this defines the extent to which the customer is satisfied with the software product delivered at the end of the development process. At a more detailed level, it involves conformance to explicitly stated requirements, explicitly documented development standards, and implicit characteristics that are expected of all professionally developed software.

software quality factor (SQF) An attribute of a software product affected by product operation requirements, product revision requirements and product transition requirements. There are 11 such attributes: correctness, reliability, efficiency, integrity, usability, maintainability, flexibility, testability, portability, reusability and interoperability.

software requirement See system requirement. See in contrast, user requirements.

software system A general term referring to the end-product of software development. It embraces both application software and system software.

specialisation A relationship between classes in the object-oriented model that captures the fact that a child class is a subtype of the parent class, that is, the methods and data fields of the specialised class (subclass) are a superset of the generalised class (superclass specialised class).

specific-requirements capture Capturing the specific requirements for a new product, when using a product-line process. These are captured as specialisations and extensions of the domain model.

sporadic event An event that is infrequent and isolated.

sprint In Scrum, a timeboxed period of to a maximum of a month where useable software is delivered.

stakeholder An individual and/or an organisation affected by the success or failure of a software system.

start-up testing System testing that tests the system's ability to be started in a working hardware/software configuration. Following such tests, all configurations should be fully system tested individually.
Also referred to as initialisation testing.

state A condition or situation during the lifetime of an object during which it satisfies some condition, performs some activity or waits for some event. In object-oriented terms, it is the value of an object's attributes. From the client's point of view, the state of an object is implemented in terms of methods (that is, its accessor methods). However, inside the object's class definition, the accessor methods may be implemented in terms of private instance variables which are used to store the state (although on occasion the implementation may be in terms of methods which calculate the state). In formal specification, it refers to the stored data that a system accesses and alters.

statechart Formalism introduced by David Harel for the representation of complex state transitions; it was adopted by the UML in its state machines.

state-dependent/independent An object that always responds in the same way to a given event is state-independent. In contrast, objects that react differently to a given event are state-dependent.

state diagram See statechart.

statelessness A property of a component when it responds to each request as a 'one-off', without retaining any memory of previous requests.

state machine A specification of the sequences of states that an object goes through in response to events during its life, together with its responsive actions. A state machine is attached to a source class, collaboration or method and specifies the behaviour of the instances of the source elements. State machines are represented in UML by state diagrams.

static analysis tools Tools that allow software or other development artefacts to be checked for some properties or tested at compile time.

static model A model that describes structure rather than behaviour. See structural model.

static modelling Modelling of structure as opposed to behaviour.

steering A move of the planning game that updates the plan based on reality.

stereotype An extension mechanism in UML; it allows the creation of a new category of modelling element.

stochastic event An event that occurs randomly but according to a well-defined probability distribution: for example heads coming up when a coin is flipped.

straight-through path Basis-path testing is a white box testing technique that involves choosing test data to exercise all of the paths in code.

In order to enumerate basis-path test cases, a first step is typically to identify the straight-through path. This is the path where all loop and if conditions evaluate to false, and only default cases are selected in switch statements.

stress testing System testing that tests that the system can operate reliably at the limits of each of its resources (for example, to stress test a web server, we may simulate the accesses of hundreds or thousands of users all at the same time to see if it can cope with the load).

structural domain model See concept model.

structural model See domain structural model, analysis structural model or design structural model.

style See architectural style.

subclass In object-oriented terms, the subclass inherits from the superclass, or parent class. In Java, all classes are subclasses of Object. A subclass may define its own methods and data fields in addition to those that are inherited. It may also override any of its inherited methods.

subdomain Simply a set of input values that require essentially the same processing. Subdomains may be found by means of case analysis.

subscriber Component, in a publish-subscribe architecture, that registers to receive updates from another component (the publisher).

substitutability Substitutability means that an object of one type is legitimately substitutable for an object of another type. This is allowed in Java when objects are related through inheritance (that is, an object of the child class may be substituted for an object of the parent class). However, substitutability is only sensible when the objects are related as subtype to supertype (that is, when the inheritance is inheritance for specialisation and not inheritance for construction).

superclass In Java, the (parent) class that is extended by another class (the child class or subclass).

supplier In software development, the term refers to the person (or company) who develops a piece of software as a package.

In programming, the term often refers to the method/class/object that is used by methods of another class/object (the client).

In network programming, the term refers to a program/process running on one computer that provides services to another program running (usually) on another computer.

swimlane A means of indicating a role's responsibilities in an activity diagram. Swimlanes show the concurrent activities of actors as separate partitions.

synchronisation bar A means of describing the coordination of activities. Once all the activities leading to the synchronisation bar have completed, the transitions leaving it will be fired. Thus, it is a point where the flow of control may change. A synchronisation bar is used to identify the points at which concurrent threads begin (a fork) or end (a join).

system An assembly of components that are connected in an organised way to perform some activity, that is, they behave dynamically.

system boundary A conceptual division between the system to be studied and 'everything else'.

system class In analysis, a class with a single instance that represents the whole system.

system independence The degree to which a system is independent of non-standard programming language features, operating system characteristics, and other environmental constraints.

system operation An operation executed by a software system as a response to an external event.

system requirement '...are a more detailed description of the functionality to be provided.' (Somerville, 2011.)

system size A method for estimating the time to complete a software project based on the number of functions, the amount of data and the number of users. At completion of the project, it can be measured by the number of lines of code (LOC or KLOC) and the volume of data.

system testing The third stage of developmental testing in which the completed system is tested against the customer's requirements.

T

tactic A reusable solution to the problem of achieving particular qualities.

tangible objects Physical objects identified during modelling.

task A single path of execution through a program, dynamic model or other representation of control flow such as an activity diagram. In general, it is a piece of work to be done by someone, such as an actor involved in a use case. It is also used as a synonym for process.

technical solution requirements These relate to functionality that is needed purely because of the chosen technology.

testability A software quality factor affecting product revision requirements which determines the effort required to test a system to ensure that it performs its intended function, i.e. how easily the system can be tested to show that the customer's requirements have been met.

test case The set of conditions and data used to determine whether a system is working as intended.

test-driven development Test driven development demands that, before any code is written, tests should be written for that code to meet. Just enough code should be written to pass those tests, and the further tests should be written and so on. On each iteration, all the tests should be run. Test driven development eases the business of making late changes to code, because the accumulated tests provide assurance that changes have not broken the code.

test harness Software added to a system to facilitate testing.

testing The process of determining if an artefact meets a given specification. It is one of the four basic technical activities within a software development process (see also analysis, design and implementation). In practice, it is the use of techniques for the validation or verification of products or the processes by which they are made.

test stub A testing tool providing very simple pieces of code which stand in for the called method. A test stub will implement the same signature as the called method, but provide none of its functionality, printing only verification messages for the data sent to it and returning to the caller with the 'expected' value.

throughput The number of events dealt with in a given time.

tier A physical partitioning of a system.

timeboxed Within a short time frame.

timeboxing Allocating short fixed periods – for example, three to four weeks – to each iteration.

to do list In Test-driven development, when coding anything that turns out to need more than two or three TDD cycles, it can be useful to maintain a short to do list that can feed into the first steps of future cycles. This list should be kept short.

tool Software used to support the software development process.

traceability The ability to trace the history of each requirement from problem description through conceptual and specification models to the implementation and deployment models in order to reconstruct the significant events that led to the final operational software system. The ability to record information that will allow any investigation to reconstruct events, changes and reasons for change.

trade 'A job, especially needing a special skill with the hands: Being a printer is a trade; being a lawyer is a profession.' (Longman Dictionary of Contemporary English, 1987.)

training A process which assists in enabling new users to use a system effectively. It may include manuals, self-study modules or face-to-face lecturing.

transition In state machines or activity diagram, the movement between states or activities triggered by an event. In UP, phase when the final product is tested and deployed.

U

UML See Unified Modelling Language.

Unified Modelling Language (UML) A language for visualising, specifying, constructing and documenting the artefacts of a software-intensive system. (See also modelling language.)

Unified Process (UP) A popular iterative and incremental process for building enterprise systems based on an object-oriented approach. It consists of four phases: inception, elaboration, construction and transition.

unit testing The first stage of developmental testing in which units of functionality (for example, the classes in an object-oriented system) are tested in isolation.

usability The effectiveness, efficiency and satisfaction with which users can achieve tasks in a particular environment. High usability means a system is easy to learn and remember; efficient, visually pleasing and fun to use; and quick to recover from errors. Also, usability is a software quality factor affecting product operation requirements which determines the effort required to learn about, to operate, to prepare input for, and to interpret the output of a system, that is, how easy the system is to use.

usability requirement A non-functional requirement about the product's ease of use or other usability considerations.

usability testing One of the four major types of software testing in which the functionality and user interface are tested to ensure that they are fit for the intended users. This generally involves selected users undertaking carefully designed tasks so that measures of usability can be calculated and fed back into the development process.

use case A description, of a piece of functionality that a system performs to yield an observable result of some value to an actor. (See also task.)

use case diagram A diagram that shows a set of use cases and actors and their associations.

use case object An object which is an instance of a class defined for each use case. In a hotel system, for example, there might be classes such as CheckerIn, CheckerOut and RoomMover. Each would have a method with a name like run(). The user interface would then create a single instance of the appropriate class, initialise it suitably, and then send it the run() message.

use case realisation An activity that consists of linking the use cases to the classes which are going to carry them through.

user A person who uses the software system on a day-to-day basis.

user-centred design During analysis and design, a way of taking into account the different perspectives of different users when looking at the tasks that they perform; typically found in techniques used for requirements engineering.

user-command testing System testing that tests all user commands from the point of view of tolerance to syntax errors and data input errors. Also referred to as operator testing.

user-defined type A data type defined by the programmer using predefined types provided by the programming language.

user requirements '... abstract statements of the software requirements for the customer and end-user of the system.' (Somerville, 2004.)
See in contrast, software requirements.

user story A story written by an intended user of a system; it describes some functionality that is of value to the person(s) writing the story. It represents a user's expectation of the system.

V

validation The process of checking that a software system is consistent with the customer's requirements; that is, the developer is building the 'right' product. (See also verification.)

value identity The identity of a real-world object; how people distinguish between objects using identifiers with values which are understood by other people.

variables In Java, a data field that holds a particular type of value or refers to a particular type of object. Unlike a data field declared as static, the value associated with a variable data field may be changed. (See also class variable and instance variable.)

variant In a configuration management system, two configuration items which are essentially the same but differ in some well-defined sense are called variants. Variants are distinguished by having different descriptive names, and also have descriptions characterising them. Variants themselves undergo revisions and thus have versions.

verification A process of ensuring that the design of a system corresponds to its specification, and that the system is correctly coded according to its design; that is, all the different system descriptions are consistent and complete with respect to each other. If validation is about building the 'right' product, then verification is about the product being built 'right'.

version An 'instance' of a configuration item. Also referred to as a revision.

view In the MVC, the view is the representation of the model in the user interface: it displays the output from the system and is what the user perceives of the model's state.

viewpoint A perspective on a system from the point of view of a particular person or group of people.

virtual machine A software simulation of a computer, running in a special environment.

visibility The property of where named elements can be accessed in a program.

Volere template This is a template for a document that collates all the requirements of a system, together with other issues that may affect those requirements. The template provides a sort of container, in which the information can be organised systematically.

W

waterfall model A particular life cycle that uses the analysis, design, implementation and testing activities in a simple sequence with little or no iteration. Some versions append the maintenance activity.

weighted-methods-per-class (WMPC) metric A measure of the complexity of behaviour of a given class. It is defined as the sum of the cyclomatic complexities of each method of the class. The relationship between this metric and complexity is that a class with many

simple methods may be as difficult to understand as a class with a few complicated methods.

white box testing Used to check that the details of an implementation are correct. White box testing ignores the 'big picture' of the requirements, and instead looks to detailed designs to check that the system does what it is supposed to do, correctly.

workflow The steps that people go through when doing their jobs. A workflow may contain a number of activities that must be coordinated and arranged in a specified order to achieve a goal. A workflow can be used to represent the flow of activities that takes place in a business process.

wrapper See Adapter pattern.

X

XP See extreme programming.